VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum, Karnataka -590014

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

PUTTA HANISHA REDDY(1BF24CS236)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Putta Hanisha Reddy(1BF24CS236), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.

The Lab report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE,Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

a) FCFS

b) SJF

c) Priority

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

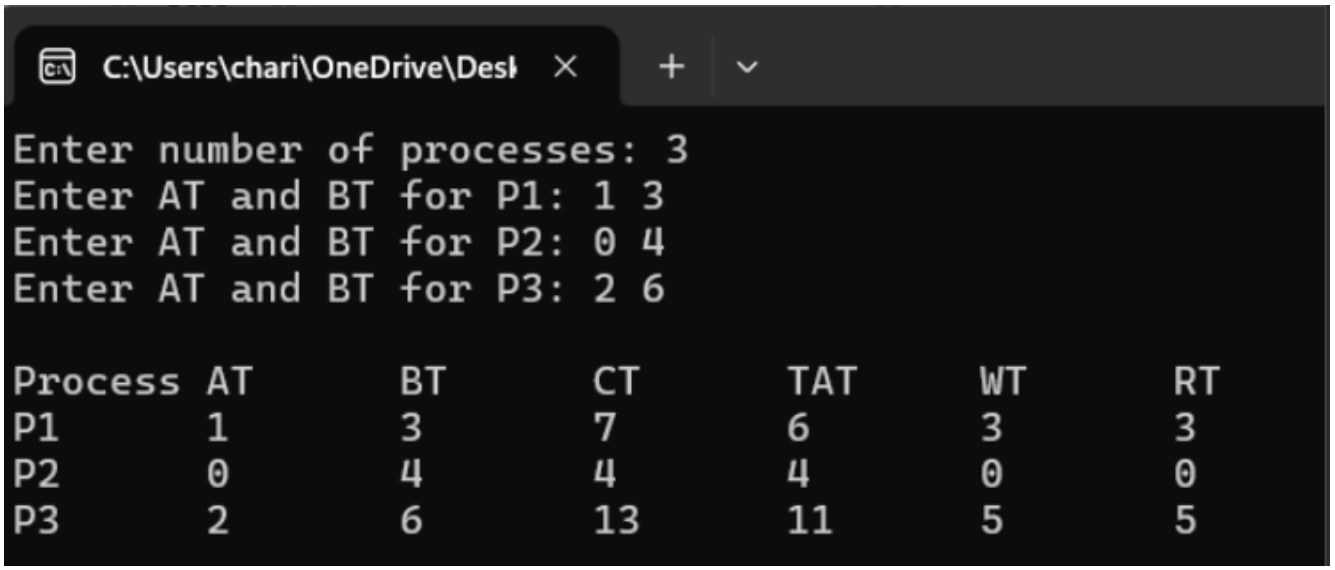
Code:

=>FCFS:

```
#include <stdio.h>

int main() {
    int n,i,j;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int pid[n],at[n],bt[n];
    int ct[n],tat[n],wt[n],rt[n];
    int start[n];
    for(i=0;i<n;i++)
    {
        pid[i]=i;
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
    }
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(at[pid[i]] > at[pid[j]])
            {
                int temp = pid[i];
                pid[i] = pid[j];
                pid[j] = temp;
            }
        }
    }
    int time = 0;
    for(i=0;i<n;i++)
    {
        int p = pid[i];
```

```
        if(time < at[p])
            time = at[p];

        start[p] = time;
        rt[p] = start[p] - at[p];

        time += bt[p];
        ct[p] = time;

        tat[p] = ct[p] - at[p];
        wt[p] = tat[p] - bt[p];
    }
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
    }
    return 0;
}
```

Result:



=>SJF :

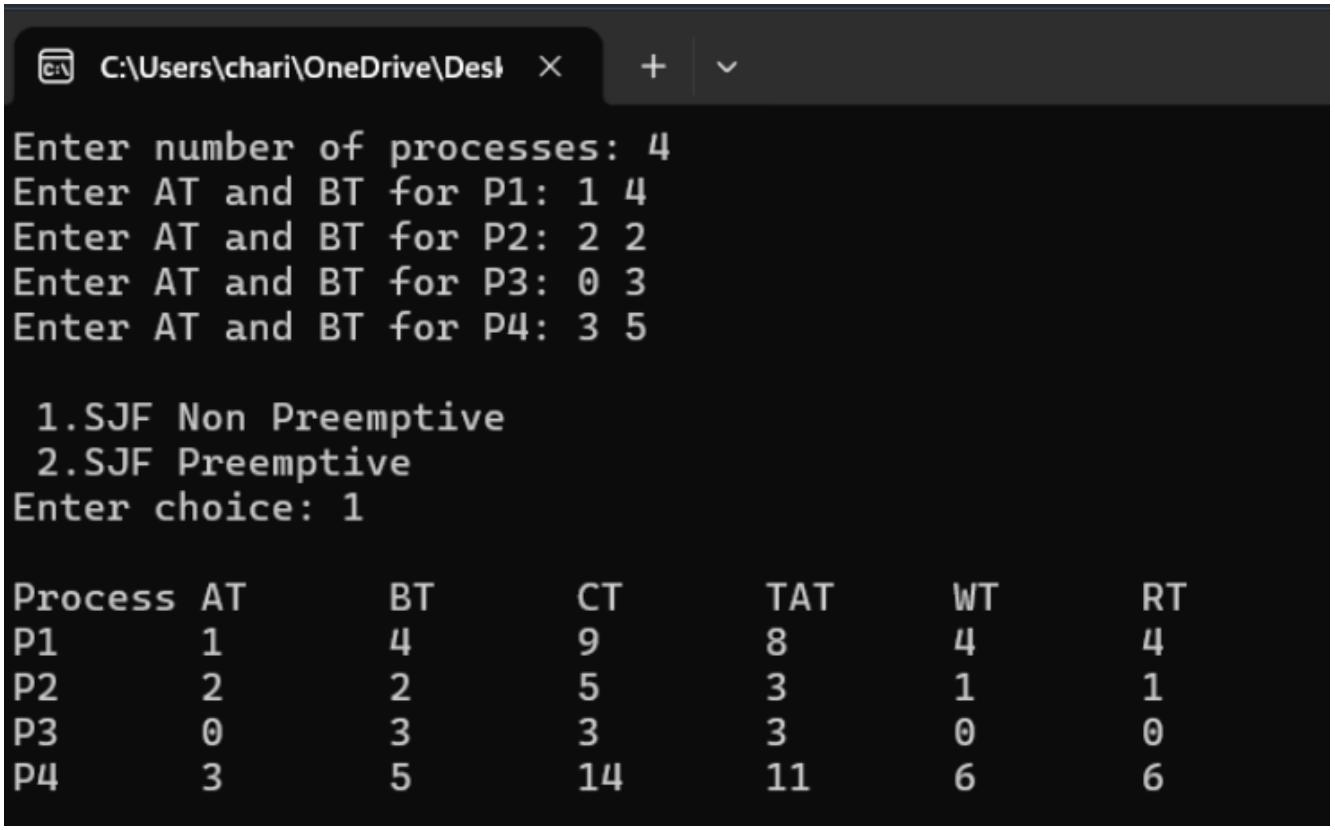
```
#include<stdio.h>
int main()
{
    int n,i,choice;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int at[n],bt[n];
    int ct[n],tat[n],wt[n],rt[n];
    for(i=0;i<n;i++)
    {
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
    }

    printf("\n 1.SJF Non Preemptive");
    printf("\n 2.SJF Preemptive");
    printf("\nEnter choice: ");
    scanf("%d",&choice);

    if(choice==1)
    {
        int time=0,completed=0;
        int visited[10]={0};
```

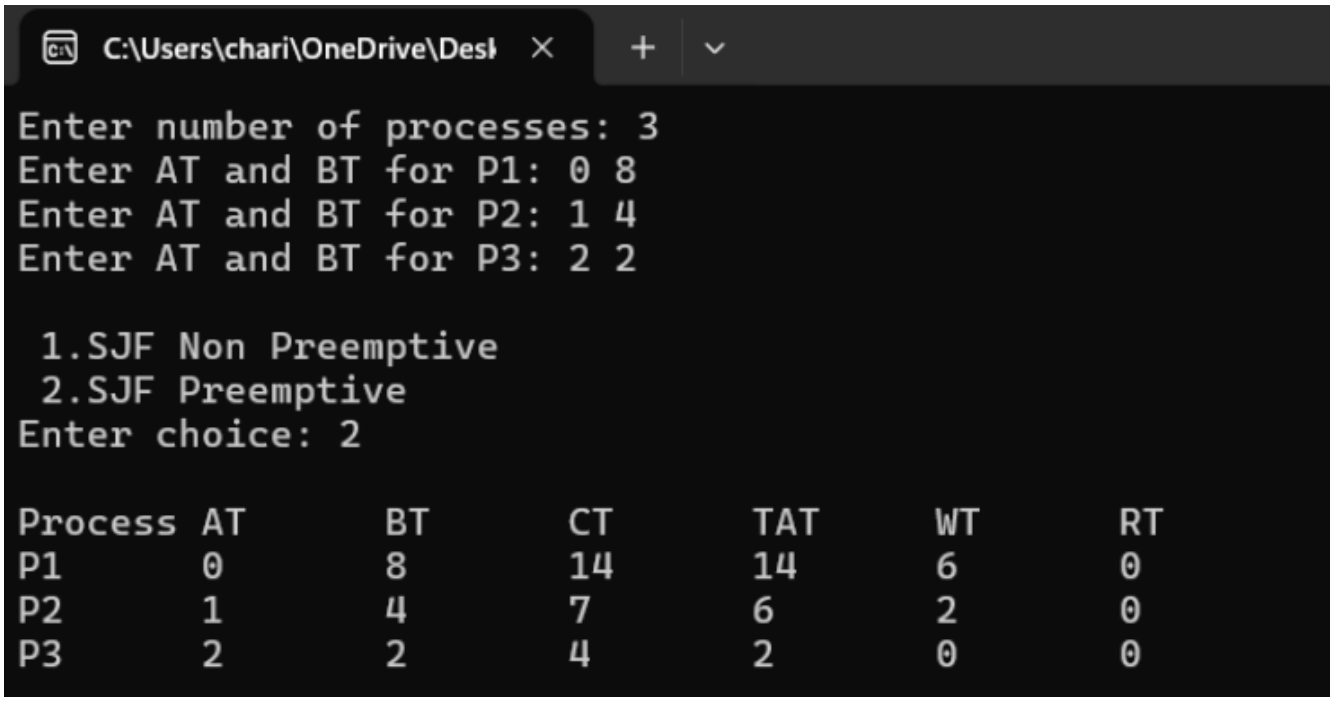
```
    while(completed<n)
    {
        int idx=-1,min=9999;
        for(i=0;i<n;i++)
        {
            if(at[i]<=time && visited[i]==0 && bt[i]<min)
            {
                min=bt[i];
                idx=i;
            }
        }
        if(idx!=-1)
        {
            rt[idx]=time-at[idx];
            time+=bt[idx];
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            visited[idx]=1;
            completed++;
        }
        else
            time++;
    }
}
```

```
else if(choice==2)
   {
      int time=0, completed=0;
      int rbt[10];
      int first[10]={0};
      for(i=0;i<n;i++)
         rbt[i]=bt[i];
      while(completed<n)
      {
         int idx=-1,min=9999;
         for(i=0;i<n;i++)
         {
            if(at[i]<=time && rbt[i]>0 && rbt[i]<min)
            {
               min=rbt[i];
               idx=i;
            }
         }
         if(idx!=-1)
         {
            if(first[idx]==0)
            {
               rt[idx]=time-at[idx];
               first[idx]=1;
            }
            rbt[idx]--;
```

```
            time++;
            if(rbt[idx]==0)
            {
                ct[idx]=time;
                tat[idx]=ct[idx]-at[idx];
                wt[idx]=tat[idx]-bt[idx];
                completed++;
            }
        }
        else
            time++;
    }
  }
  printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
  for(i=0;i<n;i++)
  {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
  }
  return 0;
}
```

Result:


C:\Users\chari\OneDrive\Desk
Enter number of processes: 4
Enter AT and BT for P1: 1 4
Enter AT and BT for P2: 2 2
Enter AT and BT for P3: 0 3
Enter AT and BT for P4: 3 5
1.SJF Non Preemptive
2.SJF Preemptive
Enter choice: 1
Process AT BT CT TAT WT RT
P1 1 4 9 8 4 4
P2 2 2 5 3 1 1
P3 0 3 3 3 0 0
P4 3 5 14 11 6 6


C:\Users\chari\OneDrive\Desk
Enter number of processes: 3
Enter AT and BT for P1: 0 8
Enter AT and BT for P2: 1 4
Enter AT and BT for P3: 2 2
1.SJF Non Preemptive
2.SJF Preemptive
Enter choice: 2
Process AT BT CT TAT WT RT
P1 0 8 14 14 6 0
P2 1 4 7 6 2 0
P3 2 2 4 2 0 0

=>Priority:

```
#include<stdio.h>

int main()
{
    int n,i,time=0,completed=0,choice;
    int at[10],bt[10],pr[10],rt[10];
    int ct[10],tat[10],wt[10],res[10];
    int start[10]={0};
    int visited[10]={0};
    float avg_tat,avg_wt,avg_rt;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter AT BT Priority for P%d: ",i+1);
        scanf("%d %d %d",&at[i],&bt[i],&pr[i]);
        rt[i]=bt[i];
        start[i]=-1;
    }
```

```
printf("\n1. Priority Non-Preemptive\n");
printf("2. Priority Preemptive\n");
printf("Enter choice: ");
scanf("%d",&choice);

printf("\nGantt Chart:\n|");

if(choice==1)
{
   while(completed<n)
   {
      int idx=-1;
      int high=999;

      for(i=0;i<n;i++)
      {
         if(at[i]<=time && visited[i]==0)
         {
            if(pr[i]<high)
            {
               high=pr[i];
               idx=i;
            }
```

```
            }
        }

        if(idx!=-1)
        {
            start[idx]=time;
            res[idx]=start[idx]-at[idx];

            printf(" P%d |",idx+1);

            time+=bt[idx];
            ct[idx]=time;

            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];

            visited[idx]=1;
            completed++;
        }
        else
        time++;
    }
}
```

```
else
{
   while(completed<n)
   {
      int idx=-1;
      int high=999;

      for(i=0;i<n;i++)
      {
         if(at[i]<=time && rt[i]>0)
         {
            if(pr[i]<high)
            {
               high=pr[i];
               idx=i;
            }
         }
      }

      if(idx!=-1)
      {
         if(start[idx]==-1)
```

```
            {
                start[idx]=time;
                res[idx]=start[idx]-at[idx];
            }

            printf(" P%d |",idx+1);

            rt[idx]--;
            time++;

            if(rt[idx]==0)
            {
                ct[idx]=time;
                tat[idx]=ct[idx]-at[idx];
                wt[idx]=tat[idx]-bt[idx];
                completed++;
            }
        }
        else
        time++;
    }
}
```

```
    printf("\n\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],res[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
    avg_rt += res[i];
}

avg_tat /= n;
avg_wt /= n;
avg_rt /= n;

printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Response Time = %.2f\n", avg_rt);
}
```

Result:

```
Enter number of processes: 3
Enter AT BT Priority for P1: 1 4 3
Enter AT BT Priority for P2: 0 1 2
Enter AT BT Priority for P3: 2 6 1

1. Priority Non-Preemptive
2. Priority Preemptive
Enter choice: 1

Gantt Chart:
| P2 | P1 | P3 |

PID     AT      BT      PR      CT      TAT     WT      RT
P1      1       4       3       5       4       0       0
P2      0       1       2       1       1       0       0
P3      2       6       1       11      9       3       3

Average Turnaround Time = 4.67
Average Waiting Time = 1.00
Average Response Time = 1.00
```

```
Enter number of processes: 3
Enter AT BT Priority for P1: 1 4 3
Enter AT BT Priority for P2: 0 1 2
Enter AT BT Priority for P3: 2 6 1

1. Priority Non-Preemptive
2. Priority Preemptive
Enter choice: 2

Gantt Chart:
| P2 | P1 | P3 | P3 | P3 | P3 | P3 | P3 | P1 | P1 | P1 |

PID     AT      BT      PR      CT      TAT     WT      RT
P1      1       4       3       11      10      6       0
P2      0       1       2       1       1       0       0
P3      2       6       1       8       6       0       0

Average Turnaround Time = 5.67
Average Waiting Time = 2.00
Average Response Time = 0.00
```

=>Round Robin:

```
#include <stdio.h>

int main() {
    int n, i, time = 0,
tq, remain;
    int at[10], bt[10],
rt[10];
    int ct[10],
tat[10], wt[10],
response[10];
    int
first_exec[10];

    // Gantt Chart
    int gantt_p[100],
gantt_t[100];
    int g_index = 0;

    float total_wt =
0, total_tat = 0,
total_rt = 0;

    printf("Enter
number of
processes: ");
    scanf("%d", &n);

    remain = n;
```

```
    for(i = 0; i < n;
i++) {
        printf("Enter
arrival time and
burst time for P%d:
", i + 1);
        scanf("%d
%d", &at[i],
&bt[i]);

        rt[i] = bt[i];
        first_exec[i] =
-1;
    }

    printf("Enter
Time Quantum: ");
    scanf("%d",
&tq);



    while(remain !=
0) {
        int done = 1;

        for(i = 0; i < n;
i++) {
            if(rt[i] > 0
```

```
&& at[i] <= time) {
            done = 0;

            //
Response Time

if(first_exec[i] == -
1) {

first_exec[i] = time;

response[i] = time -
at[i];
            }

gantt_p[g_index] =
i;

gantt_t[g_index] =
time;

g_index++;

            if(rt[i] >
tq) {
               time
+= tq;
              rt[i] -=
tq;
            } else {
```

```
                time
+= rt[i];
                ct[i] =
time;
                rt[i] =
0;

remain--;
            }
        }
    }

    if(done) {
        time++;
    }
  }

  gantt_t[g_index]
= time;

  // Calculations
  for(i = 0; i < n;
i++) {
    tat[i] = ct[i] -
at[i];
    wt[i] = tat[i] -
bt[i];
    total_tat +=
tat[i];
    total_wt +=
```

```
wt[i];
        total_rt +=
response[i];
    }

    // Gantt Chart
    printf("\nGantt
Chart:\n");

    printf("|");
    for(i = 0; i <
g_index; i++) {
        printf(" P%d
|", gantt_p[i] + 1);
    }

    printf("\n0");
    for(i = 0; i <
g_index; i++) {
        printf("    %d",
gantt_t[i + 1]);
    }

    printf("\n");

    // Output Table
printf("\n\nProcess\t
AT\tBT\tCT\tTAT\t
WT\tRT\n");
    for(i = 0; i < n;
```

```
i++) {

printf("P%d\t%d\t
%d\t%d\t%d\t%d\t
%d\n",
            i + 1, at[i],
bt[i], ct[i], tat[i],
wt[i], response[i]);
    }


printf("\nAverage
TAT = %.2f",
total_tat / n);

printf("\nAverage
WT = %.2f",
total_wt / n);

printf("\nAverage
RT = %.2f\n",
total_rt / n);

    return 0;
}
```

Result:

```
Enter number of processes: 3
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 4
Enter arrival time and burst time for P3: 2 7
Enter Time Quantum: 3

Gantt Chart:
| P1 | P2 | P3 | P1 | P2 | P3 | P3 |
0    3    6    9    11    12    15    16


Process AT      BT      CT      TAT     WT      RT
P1      0       5       11      11      6       0
P2      1       4       12      11      7       2
P3      2       7       16      14      7       4

Average TAT = 12.00
Average WT = 6.67
Average RT = 2.00
```

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

=>Multilevel:

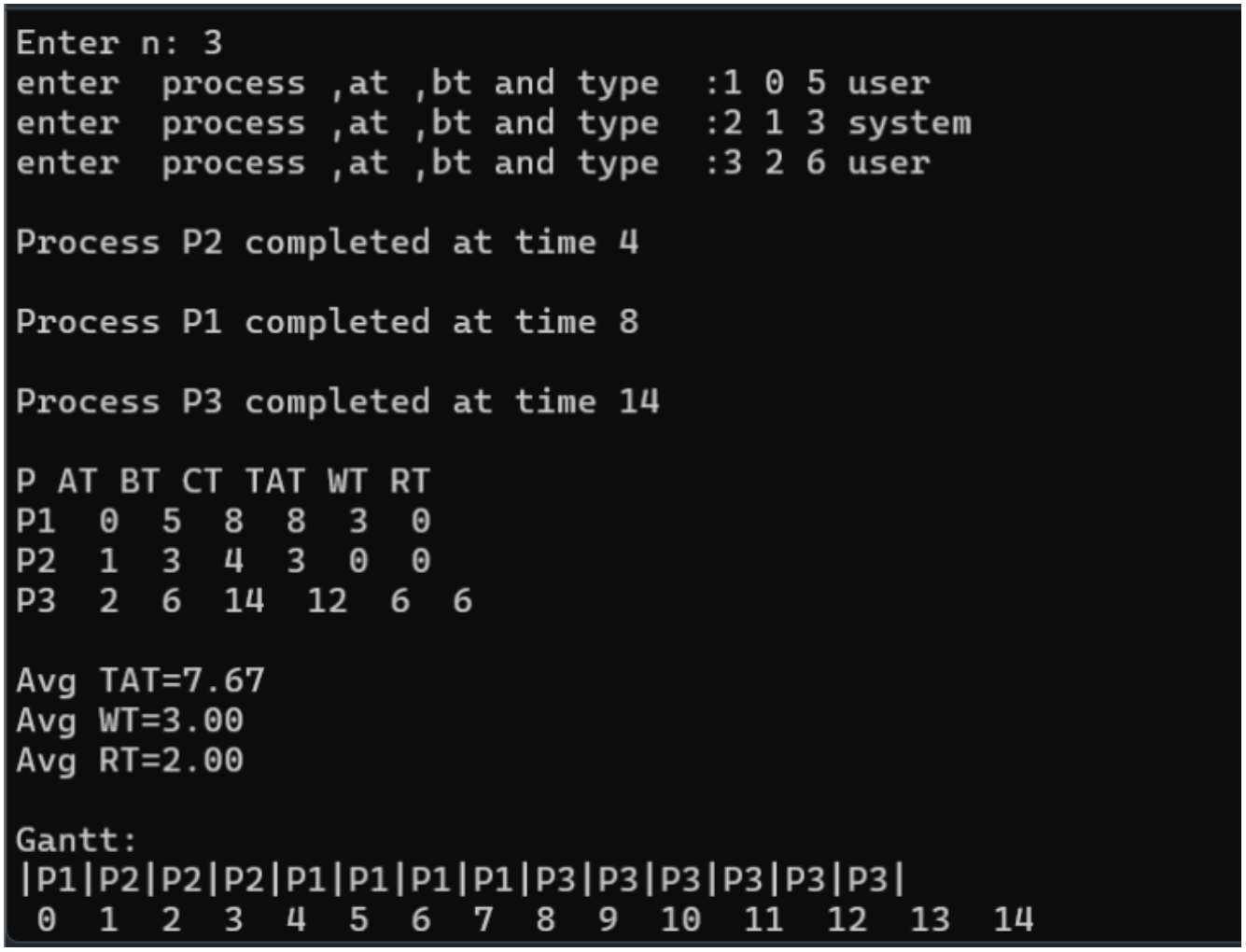
```
#include <stdio.h>
#include <string.h>

#define MAX 20

int main() {
   int n,i,j,time=0,comp=0,idx=0,cur=-1;
   int pid[MAX],at[MAX],bt[MAX],rt[MAX],ct[MAX],tat[MAX],wt[MAX],rtm[MAX],fe[MAX];
   char type[MAX][10];

   int sq[MAX],uq[MAX],sf=0,sr=-1,uf=0,ur=-1;
   int gp[200],gt[200],g=0;

   printf("Enter n: "); scanf("%d",&n);
   for(i=0;i<n;i++){
      printf("enter  process ,at ,bt and type  :",i+1);
      scanf("%d%d%d%s",&pid[i],&at[i],&bt[i],type[i]);
      rt[i]=bt[i]; fe[i]=-1;
   }

   // sort by arrival time
   for(i=0;i<n-1;i++)
      for(j=i+1;j<n;j++)
         if(at[i]>at[j]){
            int t;
            t=at[i];at[i]=at[j];at[j]=t;
            t=bt[i];bt[i]=bt[j];bt[j]=t;
            t=rt[i];rt[i]=rt[j];rt[j]=t;
            t=pid[i];pid[i]=pid[j];pid[j]=t;
```

```
            char tmp[10]; strcpy(tmp,type[i]); strcpy(type[i],type[j]); strcpy(type[j],tmp);
        }

while(comp < n){

    // add arrived processes
    while(idx<n && at[idx]<=time){
        if(strcmp(type[idx],"system")==0)
            sq[++sr] = idx;
        else
            uq[++ur] = idx;
        idx++;
    }

    // preemption
    if(cur!=-1 && strcmp(type[cur],"user")==0 && sf<=sr){
        uq[++ur] = cur;
        cur = -1;
    }

    // select process
    if(cur == -1){
        if(sf <= sr){
            cur = sq[sf++];
        }
        else if(uf <= ur){
            cur = uq[uf++];
        }
        else{
            time++;
            continue; // ✅ now valid (inside loop)
        }
    }

    // response time
    if(fe[cur]==-1){
        fe[cur]=time;
        rtm[cur]=time-at[cur];
    }
```

```
        // gantt
        gp[g]=cur;
        gt[g++]=time;

        // execute
        rt[cur]--;
        time++;

        // completion
        if(rt[cur]==0){
    ct[cur]=time;

    // ✅ Print completion message
    printf("\nProcess P%d completed at time %d\n", pid[cur], time);

    comp++;
    cur=-1;
}
    }

    gt[g]=time;

    float awt=0,atat=0,art=0;

    printf("\nP AT BT CT TAT WT RT\n");
    for(i=0;i<n;i++){
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];
        awt+=wt[i]; atat+=tat[i]; art+=rtm[i];

        printf("P%d  %d  %d  %d  %d  %d  %d\n",pid[i],at[i],bt[i],ct[i],tat[i],wt[i],rtm[i]);
    }

    printf("\nAvg TAT=%.2f\nAvg WT=%.2f\nAvg RT=%.2f\n",atat/n,awt/n,art/n);

    printf("\nGantt:\n|");
    for(i=0;i<g;i++) printf("P%d|",pid[gp[i]]);
    printf("\n %d",gt[0]);
    for(i=1;i<=g;i++) printf("  %d",gt[i]);
```

```
    return 0;
}
```

Result:



```
Enter n: 3
enter  process ,at ,bt and type  :1 0 5 user
enter  process ,at ,bt and type  :2 1 3 system
enter  process ,at ,bt and type  :3 2 6 user

Process P2 completed at time 4

Process P1 completed at time 8

Process P3 completed at time 14

P AT BT CT TAT WT RT
P1  0  5  8  8  3  0
P2  1  3  4  3  0  0
P3  2  6  14  12  6  6

Avg TAT=7.67
Avg WT=3.00
Avg RT=2.00

Gantt:
|P1|P2|P2|P2|P1|P1|P1|P1|P3|P3|P3|P3|P3|P3|
 0  1  2  3  4  5  6  7  8  9  10  11  12  13  14
```

Program -3:
Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
    int n, i, j, temp, time;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int b[n], d[n], p[n], id[n];

    for (i = 0; i < n; i++) {
        id[i] = i;
        printf("\nProcess %d:\n", i);
        printf("Burst Time: "); scanf("%d", &b[i]);
        printf("Deadline: "); scanf("%d", &d[i]);
        printf("Period: "); scanf("%d", &p[i]);
    }

    float util = 0;
    for (i = 0; i < n; i++) util += (float)b[i] / p[i];

    // EDF (sort by deadline)
    for (i = 0; i < n-1; i++)
        for (j = i+1; j < n; j++)
            if (d[i] > d[j]) {
                temp=d[i]; d[i]=d[j]; d[j]=temp;
                temp=b[i]; b[i]=b[j]; b[j]=temp;
                temp=id[i]; id[i]=id[j]; id[j]=temp;
            }

    printf("\nEDF Scheduling\nCPU Utilization: %.2f\n", util);
    printf("ID  CT  WT  TAT\n");
```

```
    time = 0;
    for (i = 0; i < n; i++) {
        time += b[i];
        printf("%d   %d   %d   %d\n", id[i], time, time-b[i], time);
    }

    // RMS (sort by period)
    for (i = 0; i < n-1; i++)
        for (j = i+1; j < n; j++)
            if (p[i] > p[j]) {
                temp=p[i]; p[i]=p[j]; p[j]=temp;
                temp=b[i]; b[i]=b[j]; b[j]=temp;
                temp=id[i]; id[i]=id[j]; id[j]=temp;
            }

    float bound = n * (pow(2, 1.0/n) - 1);

    printf("\nRMS Scheduling\nCPU Utilization: %.2f\nRM Bound: %.4f\n", util, bound);
    printf("ID  CT  WT  TAT\n");

    time = 0;
    for (i = 0; i < n; i++) {
        time += b[i];
        printf("%d   %d   %d   %d\n", id[i], time, time-b[i], time);
    }

    // ---------------- Proportional ----------------
    printf("\n====== Proportional Scheduling ======\n");
    printf("ID  Burst  Share\n");
    int total=0;
    for (int i = 0; i < n; i++)
        total += b[i];
    for (int i = 0; i < n; i++) {
        float share = (float)b[i] / total;
        printf("%d   %d     %.2f\n", id[i], b[i], share);
    }
    return 0;
}
```

Result:

```
Enter number of processes: 3

Process 0:
Burst Time: 3
Deadline: 7
Period: 20

Process 1:
Burst Time: 2
Deadline: 4
Period: 5

Process 2:
Burst Time: 2
Deadline: 8
Period: 10

EDF Scheduling
CPU Utilization: 0.75
ID  CT  WT  TAT
1   2   0   2
0   5   2   5
2   7   5   7

RMS Scheduling
CPU Utilization: 0.75
RM Bound: 0.7798
ID  CT  WT  TAT
0   3   0   3
2   5   3   5
1   7   5   7

====== Proportional Scheduling ======
ID  Burst  Share
0   3      0.43
2   2      0.29
1   2      0.29
```

Program -4
Question:

Write a C program to simulate:
a) Producer-Consumer problem using semaphores.
b) Dining-Philosopher's problem

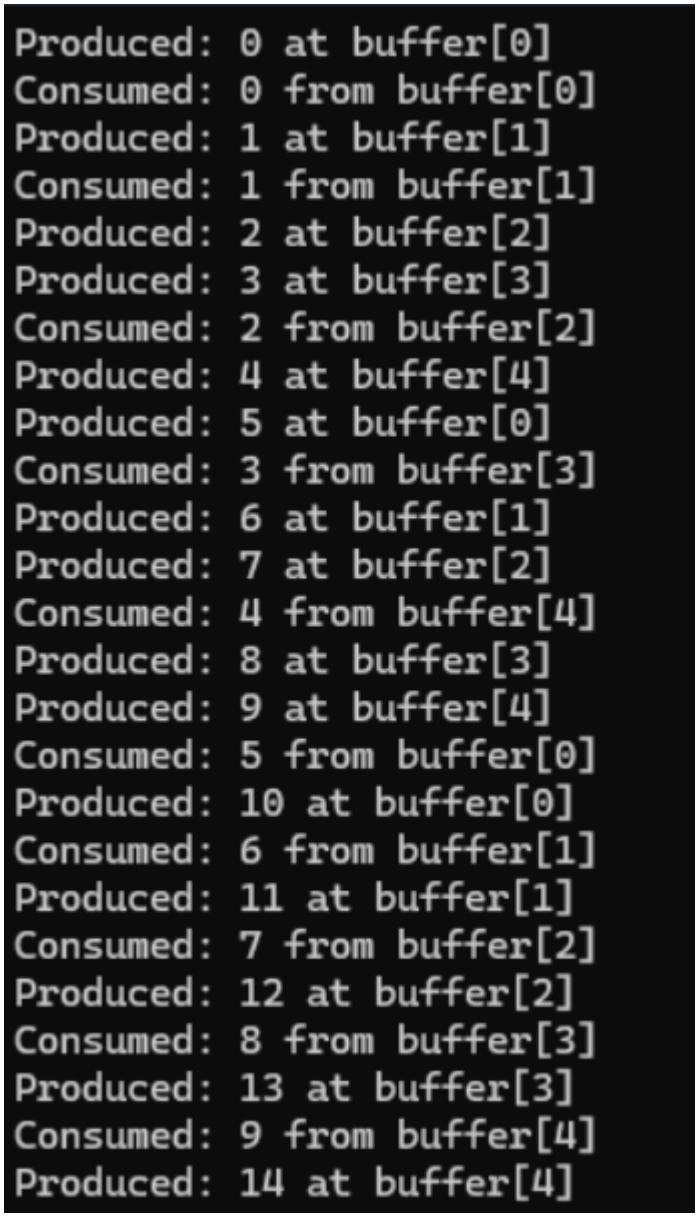
code:

=>Producer-Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;
int item = 0;

sem_t empty, full, mutex;

void *producer(void *arg)
{
   for(int i = 0; i < 15; i++)
   {
      sem_wait(&empty);
      sem_wait(&mutex);

      buffer[in] = item;
      printf("Produced: %d at buffer[%d]\n", item, in);

      item++;
      in = (in + 1) % SIZE;

      sem_post(&mutex);
      sem_post(&full);

      sleep(1);
   }
```

```
    return NULL;
}

void *consumer(void *arg)
{
    for(int i = 0; i < 10; i++)
    {
        sem_wait(&full);
        sem_wait(&mutex);

        printf("Consumed: %d from buffer[%d]\n",
              buffer[out], out);

        out = (out + 1) % SIZE;

        sem_post(&mutex);
        sem_post(&empty);

        sleep(2);
    }
    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}
```

Result:



=>Dining Philosopher

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
#define EAT_COUNT 3

sem_t forks[N];
```

```
void *philosopher(void *arg)
{
   int id = *(int *)arg;

   for(int k = 1; k <= EAT_COUNT; k++)
   {
      printf("Philosopher %d is thinking.\n", id);
      sleep(1);

      /* Deadlock-free */
      if(id == N - 1)
      {
         sem_wait(&forks[(id + 1) % N]);
         printf("Philosopher %d picked up right fork %d.\n",
               id, (id + 1) % N);

         sem_wait(&forks[id]);
         printf("Philosopher %d picked up left fork %d.\n",
               id, id);
      }
      else
      {
         sem_wait(&forks[id]);
         printf("Philosopher %d picked up left fork %d.\n",
               id, id);

         sem_wait(&forks[(id + 1) % N]);
         printf("Philosopher %d picked up right fork %d.\n",
               id, (id + 1) % N);
      }

      printf("Philosopher %d is eating \n",
            id);
      sleep(2);

      sem_post(&forks[id]);
      sem_post(&forks[(id + 1) % N]);

      printf("Philosopher %d put down forks %d and %d.\n",
            id, id, (id + 1) % N);
   }

   printf("Philosopher %d has finished all %d meals.\n",
         id, EAT_COUNT);

   return NULL;
```

```
}

int main()
{
   pthread_t ph[N];
   int id[N];

   for(int i = 0; i < N; i++)
      sem_init(&forks[i], 0, 1);

   for(int i = 0; i < N; i++)
   {
      id[i] = i;
      pthread_create(&ph[i], NULL, philosopher, &id[i]);
   }

   for(int i = 0; i < N; i++)
      pthread_join(ph[i], NULL);

   printf("\nAll philosophers have eaten %d times and left the table.\n",
         EAT_COUNT);

   for(int i = 0; i < N; i++)
      sem_destroy(&forks[i]);

   return 0;
}
```

Result:

```
Philosopher 1 is thinking.
Philosopher 0 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 2 picked up left fork 2.
Philosopher 0 picked up left fork 0.
Philosopher 3 picked up left fork 3.
Philosopher 3 picked up right fork 4.
Philosopher 3 is eating
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 picked up right fork 4.
Philosopher 3 is eating
Philosopher 1 picked up right fork 2.
Philosopher 1 is eating
Philosopher 3 put down forks 3 and 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating
Philosopher 3 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
```

Program -5
Question:
Write a C program to simulate: (Any one)
a) Bankers' algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

code:

=>Bankers' algorithm

```
#include <stdio.h>

int main() {

    int n, m;
    int i, j, k;

    printf("Enter number of processes and resources:\n");
    scanf("%d%d", &n, &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    // Calculate Need Matrix
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
```

```
// ---------------- RESOURCE REQUEST ----------------

int p;
int request[m];

printf("Enter process number requesting resources:\n");
scanf("%d", &p);

printf("Enter request vector:\n");
for(i = 0; i < m; i++)
   scanf("%d", &request[i]);

// Check Request <= Need
for(i = 0; i < m; i++) {
   if(request[i] > need[p][i]) {
      printf("Error: Process exceeded maximum claim\n");
      return 0;
   }
}

// Check Request <= Available
for(i = 0; i < m; i++) {
   if(request[i] > avail[i]) {
      printf("Resources not available. Process must wait.\n");
      return 0;
   }
}

// Pretend allocation
for(i = 0; i < m; i++) {
   avail[i] -= request[i];
   alloc[p][i] += request[i];
   need[p][i] -= request[i];
}

// ---------------- SAFETY ALGORITHM ----------------

int finish[n], safe[n];
int work[m];

for(i = 0; i < m; i++)
   work[i] = avail[i];
```

```
for(i = 0; i < n; i++)
   finish[i] = 0;

int count = 0;

while(count < n) {

   int found = 0;

   for(i = 0; i < n; i++) {

      if(finish[i] == 0) {

         for(j = 0; j < m; j++) {
            if(need[i][j] > work[j])
               break;
         }

         // Process can execute
         if(j == m) {

            for(k = 0; k < m; k++)
               work[k] += alloc[i][k];

            safe[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
   }

   if(found == 0)
      break;
}

// ---------------- RESULT ----------------

if(count == n) {

   printf("Request can be granted.\n");
   printf("System is in SAFE state.\n");

   printf("Safe Sequence: ");
```

```
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);

        printf("\n");
    }
    else {

        // Rollback
        for(i = 0; i < m; i++) {
            avail[i] += request[i];
            alloc[p][i] -= request[i];
            need[p][i] += request[i];
        }

        printf("Request cannot be granted.\n");
        printf("System would become UNSAFE.\n");
    }

    return 0;
}
```

Result:

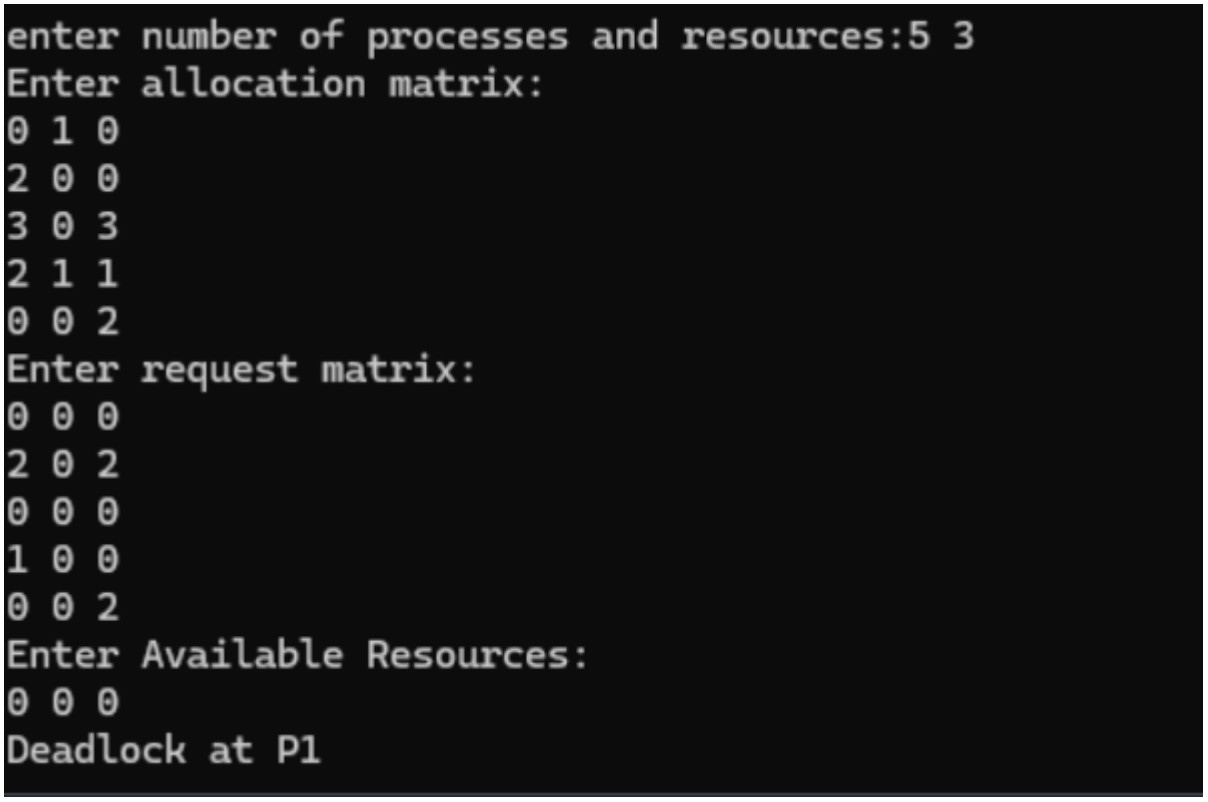
```
Enter number of processes and resources:
5 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
Enter process number requesting resources:
1
Enter request vector:
1 0 2
Request can be granted.
System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
```

=>Deadlock Detection

```
#include <stdio.h>

int main() {
   int n,m,i,j,k;
   printf("enter number of processes and resources:");
   scanf("%d%d",&n,&m);

   int alloc[n][m], req[n][m];
   int avail[m], finish[n];
   printf("Enter allocation matrix:\n");
   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         scanf("%d",&alloc[i][j]);
   printf("Enter request matrix:\n");
   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         scanf("%d",&req[i][j]);
   printf("Enter Available Resources:\n");
   for(i=0;i<m;i++)
      scanf("%d",&avail[i]);

   for(i=0;i<n;i++)
      finish[i]=0;

   for(i=0;i<n;i++) {
      if(!finish[i]) {

         for(j=0;j<m;j++)
            if(req[i][j]>avail[j])
               break;

         if(j==m) {
            for(k=0;k<m;k++)
               avail[k]+=alloc[i][k];

            finish[i]=1;
         }
      }
   }

   for(i=0;i<n;i++) {
      if(!finish[i])
         printf("Deadlock at P%d\n",i);
   }
```

```
    return 0;
}
```

Result:



Program 6

Question:

Write a C program to simulate the following contiguous memory allocation technique

a) Worst-fit b) Best-fit c) First-fit

Code:

```
#include <stdio.h>

int main() {
    int nb, np, i, j;
    int b[20], p[20], allocation[20];

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    printf("Enter sizes of %d memory blocks:\n", nb);
    for (i = 0; i < nb; i++)
        scanf("%d", &b[i]);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter sizes of %d processes:\n", np);
    for (i = 0; i < np; i++)
        scanf("%d", &p[i]);

    // ---------- FIRST FIT ----------
    int temp[20];
    for (i = 0; i < nb; i++) temp[i] = b[i];
    for (i = 0; i < np; i++) allocation[i] = -1;

    for (i = 0; i < np; i++) {
        for (j = 0; j < nb; j++) {
            if (temp[j] >= p[i]) {
                allocation[i] = j + 1;
                temp[j] = -1;
                break;
            }
        }
    }
```

```
printf("\n--- First Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (i = 0; i < np; i++) {
   if (allocation[i] != -1)
      printf("%d\t\t%d\t\t%d\n", i + 1, p[i], allocation[i]);
   else
      printf("%d\t\t%d\t\tNot Allocated\n", i + 1, p[i]);
}

// ---------- BEST FIT ----------
for (i = 0; i < nb; i++) temp[i] = b[i];
for (i = 0; i < np; i++) allocation[i] = -1;

for (i = 0; i < np; i++) {
   int bestIdx = -1;
   for (j = 0; j < nb; j++) {
      if (temp[j] >= p[i]) {
         if (bestIdx == -1 || temp[j] < temp[bestIdx])
            bestIdx = j;
      }
   }
   if (bestIdx != -1) {
      allocation[i] = bestIdx + 1;
      temp[bestIdx] = -1;
   }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (i = 0; i < np; i++) {
   if (allocation[i] != -1)
      printf("%d\t\t%d\t\t%d\n", i + 1, p[i], allocation[i]);
   else
      printf("%d\t\t%d\t\tNot Allocated\n", i + 1, p[i]);
}

// ---------- WORST FIT ----------
for (i = 0; i < nb; i++) temp[i] = b[i];
for (i = 0; i < np; i++) allocation[i] = -1;

for (i = 0; i < np; i++) {
   int worstIdx = -1;
   for (j = 0; j < nb; j++) {
      if (temp[j] >= p[i]) {
         if (worstIdx == -1 || temp[j] > temp[worstIdx])
            worstIdx = j;
```

```
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx + 1;
            temp[worstIdx] = -1;
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (i = 0; i < np; i++) {
        if (allocation[i] != -1)
            printf("%d\t\t%d\t\t%d\n", i + 1, p[i], allocation[i]);
        else
            printf("%d\t\t%d\t\tNot Allocated\n", i + 1, p[i]);
    }

    return 0;
}
```

Result:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- Best Fit ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated
```

Program 7

Question:

Write a C program to simulate page replacement algorithms.

a) FIFO b) LRU c) Optimal

Code:

```
#include <stdio.h>

int frames[10];

/* Function to check if page exists in frames */
int search(int key, int frameCount) {
    for (int i = 0; i < frameCount; i++) {
        if (frames[i] == key)
            return 1;
    }
    return 0;
}

/* FIFO Page Replacement */
void FIFO(int pages[], int n, int frameCount) {
    int pageFaults = 0, index = 0;

    for (int i = 0; i < frameCount; i++)
        frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");

    for (int i = 0; i < n; i++) {
        if (!search(pages[i], frameCount)) {
            frames[index] = pages[i];
            index = (index + 1) % frameCount;
            pageFaults++;
        }

        printf("Page %d -> ", pages[i]);
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
        printf("\n");
    }

    printf("Total Page Faults = %d\n", pageFaults);
```

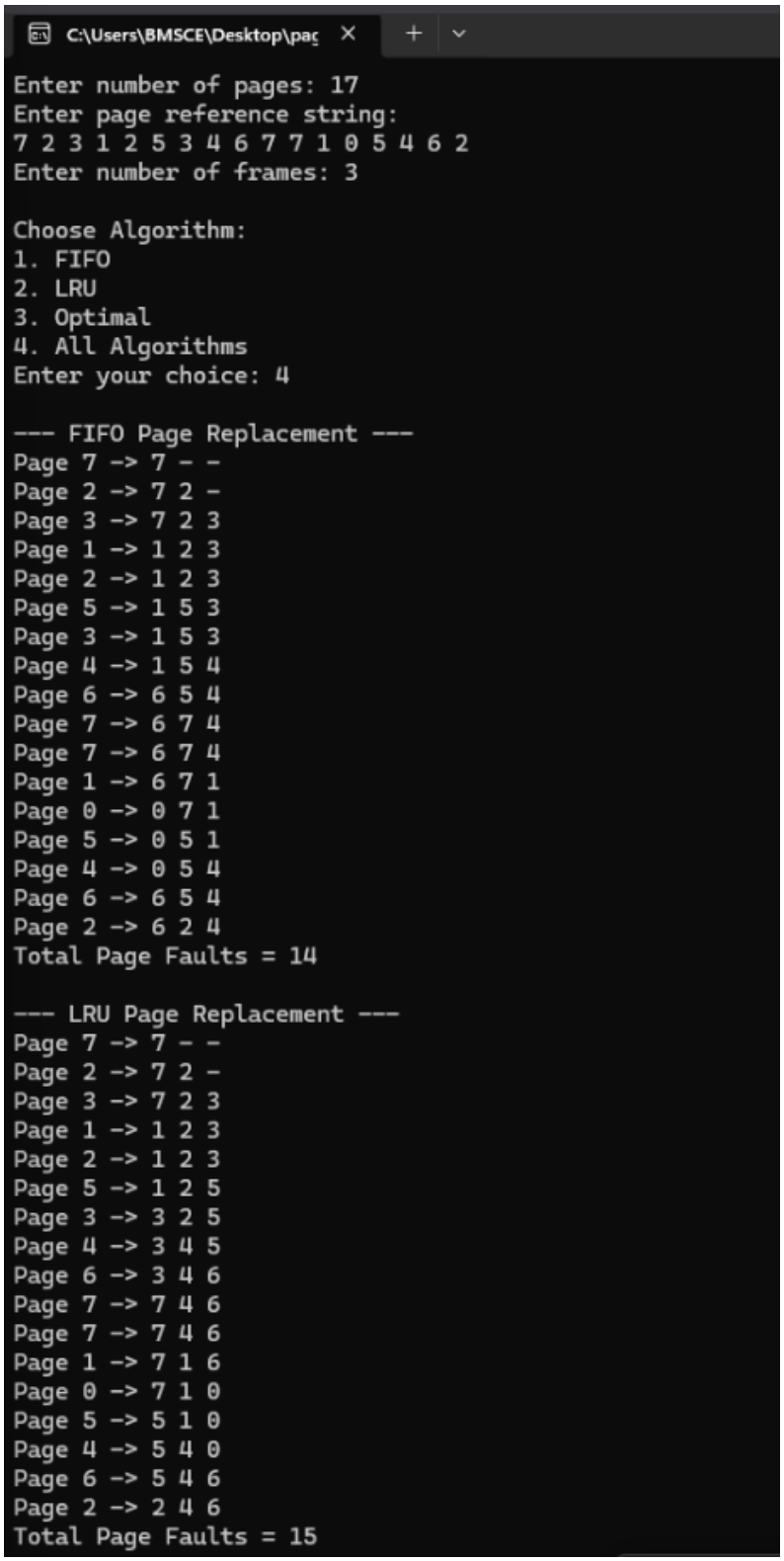
```
}

/* LRU Page Replacement */
void LRU(int pages[], int n, int frameCount) {
    int pageFaults = 0, time[10], counter = 0;

    for (int i = 0; i < frameCount; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for (int i = 0; i < n; i++) {
        int found = 0;

        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        if (!found) {
            int pos = 0;

            for (int j = 1; j < frameCount; j++) {
                if (time[j] < time[pos])
                    pos = j;
            }

            frames[pos] = pages[i];
            counter++;
            time[pos] = counter;
            pageFaults++;
        }

        printf("Page %d -> ", pages[i]);
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
        printf("\n");
    }

    printf("Total Page Faults = %d\n", pageFaults);
}

/* Optimal Page Replacement */
void Optimal(int pages[], int n, int frameCount) {
```

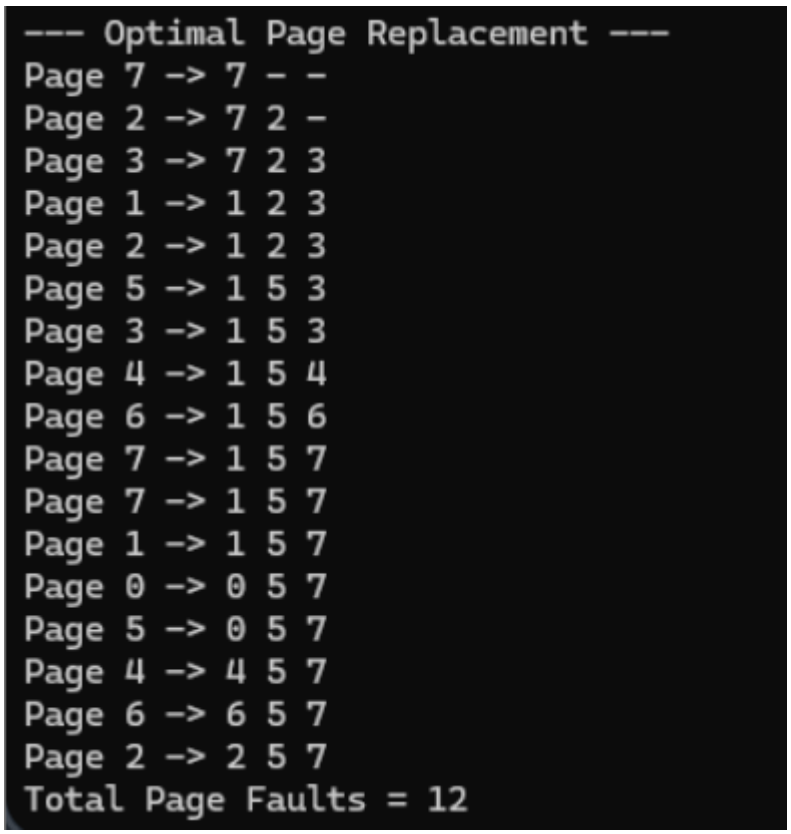
```
int pageFaults = 0;

for (int i = 0; i < frameCount; i++)
   frames[i] = -1;

printf("\n--- Optimal Page Replacement ---\n");

for (int i = 0; i < n; i++) {

   if (search(pages[i], frameCount)) {
      printf("Page %d -> ", pages[i]);

      for (int j = 0; j < frameCount; j++) {
         if (frames[j] != -1)
            printf("%d ", frames[j]);
         else
            printf("- ");
      }
      printf("\n");

      continue;
   }

   int pos = -1, farthest = i + 1;

   for (int j = 0; j < frameCount; j++) {
      int k;

      for (k = i + 1; k < n; k++) {
         if (frames[j] == pages[k]) {
            if (k > farthest) {
               farthest = k;
               pos = j;
            }
            break;
         }
      }

      if (k == n) {
         pos = j;
         break;
      }
   }

   if (pos == -1)
      pos = 0;

   frames[pos] = pages[i];
   pageFaults++;

   printf("Page %d -> ", pages[i]);

   for (int j = 0; j < frameCount; j++) {
      if (frames[j] != -1)
```

```
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
  }

  printf("Total Page Faults = %d\n", pageFaults);
}

int main() {
  int pages[50], n, frameCount, choice;

  printf("Enter number of pages: ");
  scanf("%d", &n);

  printf("Enter page reference string:\n");
  for (int i = 0; i < n; i++)
    scanf("%d", &pages[i]);

  printf("Enter number of frames: ");
  scanf("%d", &frameCount);

  printf("\nChoose Algorithm:");
  printf("\n1. FIFO");
  printf("\n2. LRU");
  printf("\n3. Optimal");
  printf("\n4. All Algorithms");
  printf("\nEnter your choice: ");
  scanf("%d", &choice);

  switch (choice) {
    case 1:
      FIFO(pages, n, frameCount);
      break;

    case 2:
      LRU(pages, n, frameCount);
      break;

    case 3:
      Optimal(pages, n, frameCount);
      break;

    case 4:
      FIFO(pages, n, frameCount);
      LRU(pages, n, frameCount);
      Optimal(pages, n, frameCount);
      break;

    default:
      printf("Invalid Choice!\n");
  }
```

Return 0;
}

Result:




--- Optimal Page Replacement ---
Page 7 -> 7 - -
Page 2 -> 7 2 -
Page 3 -> 7 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 5 -> 1 5 3
Page 3 -> 1 5 3
Page 4 -> 1 5 4
Page 6 -> 1 5 6
Page 7 -> 1 5 7
Page 7 -> 1 5 7
Page 1 -> 1 5 7
Page 0 -> 0 5 7
Page 5 -> 0 5 7
Page 4 -> 4 5 7
Page 6 -> 6 5 7
Page 2 -> 2 5 7
Total Page Faults = 12